

National University of Computer and Emerging Sciences
Lahore Campus

Data Structures (CS2001)

Date: 24th Sept 2025

Course Instructor(s)

Zareen, Waqas R, Uzair, Naveed, Arooj, Razi, and
Faisal

Sessional-1

Total Time (Hrs): 1

Total Marks: 20

Total Questions: 3

Roll No

Section

Student Signature

Instruction: Solve all the questions on this question-paper. The use of exam sheets is not allowed.

Attempt all the questions.

CLO # 2: Evaluate different data structures in terms of memory complexity and time.

Q 1: [5 marks]: Analyze the following program and determine its time complexity. For each line (or block) of code, clearly state its cost $T(n)$. Show how you derive the overall time function $T(n)$, and determine the tight Big-O complexity for both the **Best-case** and **Worst-case** scenarios. Explain your reasoning briefly in one line.

Algorithm	Cost Analysis $T(n)$
<pre>1 void Function (int arr[], int n, int pivot) { 2 int left = 0, right = n - 1; // $O(1)$ 3 4 while (left < right) { // At most $O(n)$ total 5 while (arr[left] < pivot && left < right) 6 left++; // Each element checked once, $O(n)$ total 7 8 while (arr[right] >= pivot && left < right) 9 right--; // Each element checked once, $O(n)$ 10 11 ↪ total 12 13 if (left < right) { 14 swap(arr[left], arr[right]); // Each swap $O(1)$ 15 left++; right--; // Constant time 16 } 17 }</pre>	Line 2: Initialization: $O(1)$ Pointer scans (both while loops): $O(n)$ Swaps (at most $n/2$ times): $O(n)$

Overall $T(n)$:

$$T(n) = O(1) + O(n) + O(n) = O(n)$$

Explanation:

Each element is visited at most once by the left or right pointer, and can be swapped at most once. Therefore, the algorithm performs a linear number of steps.

Best case: When all elements are already partitioned correctly.

- Left pointer scans through n elements: $O(n)$
- Right pointer stops quickly each time: $O(1)$
- Outer loop runs only once: $O(1)$

So, $T_{\text{best}}(n) = O(n)$.

Worst case: When many swaps occur (for example alternating elements around the pivot).

- Outer loop runs about $n/2$ times
- Each iteration includes constant work plus pointer movement
- Every element is still visited once in total

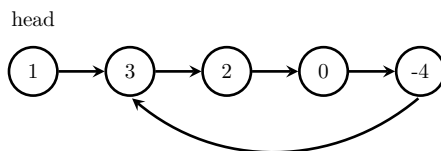
So, $T_{\text{worst}}(n) = O(n)$.

CLO 1: Demonstrate basic concepts of data structure and algorithms.

Q 2: [3 marks]: A cycle exists in a singly linked list if a node's **next** pointer points back to a previous node in the list, as shown in the figure below. You are given the head pointer of a **singly linked list**. Explain in two to three sentences how you can detect whether a cycle exists **using only a single pointer**.

Constraints:

- You are **not** allowed to use two pointers.
- You may use extra memory.
- Your solution must run in $O(n)$ time and use at most $O(n)$ space, where n is the total number of elements in the list.



Your explanation:

We can use an auxiliary boolean array where each index represents whether a node at that memory location (or with a specific ID) has been visited. As we traverse with a single pointer, we check in $O(1)$ time if the current node's corresponding array position is marked as visited. If yes, cycle exists. This gives $O(n)$ time (each node visited once with $O(1)$ check) and $O(n)$ space for the boolean array.

Detailed approach:

- (a) Create a large boolean array `visited[]` initialized to false
- (b) Map each node's memory address to an array index (using modulo or direct mapping if addresses are sequential)
- (c) Traverse the list with a single pointer
- (d) For each node, check if `visited[node_index]` is true
- (e) If true, cycle detected; if false, mark it as true and continue

National University of Computer and Emerging Sciences
Lahore Campus

CLO 1: Demonstrate basic concepts of data structure and algorithms.

Q 3: [12 marks]: Implement a C++ function `ReorderLinkedList()` in the `DoublyLinkedList` class that rearranges the list in the pattern specified below. Clearly explain your logic using proper comments (note: comments carry 2 marks).

$$L_0 \leftrightarrow L_n \leftrightarrow L_1 \leftrightarrow L_{n-1} \leftrightarrow L_2 \leftrightarrow L_{n-2} \leftrightarrow \dots$$

Here L_i represents the i th element of the list.

Example:

Input: $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5$

Output: $1 \leftrightarrow 5 \leftrightarrow 2 \leftrightarrow 4 \leftrightarrow 3$

Constraints:

- The list must be reordered entirely **in-place** (i.e., only by changing the **next** and **prev** pointers of existing nodes). Do not create a new list.
- Do not modify the data stored inside any node. Do not add new nodes or delete existing nodes.
- The algorithm should run in Time Complexity $O(n)$ and Space Complexity $O(1)$, where n is the number of elements in the list.
- If you use any helper function (e.g., **insert**, **move**, etc.), you must provide its implementation as part of your solution.
- Assume the doubly linked list does not use a sentinel (dummy) node.

Solution:

There are multiple correct ways to solve this problem since we are working with a *doubly linked list*. Both approaches below achieve the required in-place reordering with $O(n)$ time and $O(1)$ space.

Approach 1 (Splice nodes from the tail, naive idea):

- (a) Traverse to the end of the list to find the **tail**.
- (b) Maintain two pointers: **front** = **head** and **back** = **tail**.
- (c) While **front** and **back** have not met or crossed:
 - Save **nextFront** = **front**→**next**.
 - Detach **back** from the end (**back**→**prev**→**next** = **nullptr**).
 - Insert **back** after **front** by updating **front**→**next** = **back**, **back**→**prev** = **front**, and linking **back**→**next** = **nextFront**.
 - Advance **front** = **nextFront**, and set **back** = **back**→**prev**.
- (d) Stop when the two pointers meet (odd n) or cross (even n).

Example trace:

For list $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5$:

$1 \mapsto 5 \mapsto 2 \mapsto 4 \mapsto 3$

Approach 2 (Reverse the second half and merge):

- (a) Use slow/fast pointers to find the middle node of the list.
- (b) Split the list into two halves: $[L_0, L_1, \dots, L_m]$ and $[L_{m+1}, \dots, L_n]$.
- (c) Reverse the second half *in-place* by swapping **next/prev** pointers.
- (d) Merge nodes alternately: take one node from the first half, then one from the reversed second half, and continue.

Example trace:

After reversing second half of $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 4 \leftrightarrow 5$, we get:

$\underbrace{1 \leftrightarrow 2 \leftrightarrow 3}_{\text{first half}} \quad \underbrace{5 \leftrightarrow 4}_{\text{reversed second half}}$

Then merging gives the same reordered sequence:

$1 \mapsto 5 \mapsto 2 \mapsto 4 \mapsto 3$

Complexity (both approaches):

- Each node is visited and re-linked at most once, hence $O(n)$ time.
- Only a constant number of extra pointers are used, so $O(1)$ space.

Conclusion: Both methods are valid, and any student who implements either approach correctly should receive full marks.

Naive splice approach

```
1 struct Node {
2     int data;
3     Node* next;
4     Node* prev;
```

```
5     Node(int val) : data(val), next(nullptr), prev(nullptr) {}
6 };
7
8 class DoublyLinkedList {
9 private:
10     Node* head;
11
12 public:
13     DoublyLinkedList() : head(nullptr) {}
14
15     // Insert node at end (helper for testing)
16     void insertEnd(int val) {
17         Node* newNode = new Node(val);
18         if (!head) {
19             head = newNode;
20             return;
21         }
22         Node* temp = head;
23         while (temp->next) temp = temp->next;
24         temp->next = newNode;
25         newNode->prev = temp;
26     }
27
28     // Naive approach: repeatedly splice from tail and insert after front
29     void ReorderLinkedList() {
30         if (!head || !head->next) return;
31
32         // Find tail
33         Node* tail = head;
34         while (tail->next) tail = tail->next;
35
36         Node* front = head;
37         Node* back = tail;
38
39         // Continue until front and back meet or cross
40         while (front && back && front != back && front->prev != back) {
41             Node* nextFront = front->next;
42             Node* prevBack = back->prev;
43
44             // Detach 'back' from the end
45             if (prevBack) prevBack->next = nullptr;
46             back->prev = nullptr;
47
48             // Insert 'back' after 'front'
49             front->next = back;
50             back->prev = front;
51             back->next = nextFront;
52             if (nextFront) nextFront->prev = back;
53
54             // Advance pointers
```

```
55         front = nextFront;
56         back  = prevBack;
57     }
58 }
59
60 void printList() {
61     Node* temp = head;
62     while (temp) {
63         cout << temp->data << " ";
64         temp = temp->next;
65     }
66     cout << endl;
67 }
68 };
```

Approach 2: Reverse the second half and merge

```
1  struct Node {
2      int data;
3      Node* next;
4      Node* prev;
5      Node(int val) : data(val), next(nullptr), prev(nullptr) {}
6  };
7
8  class DoublyLinkedList {
9  private:
10     Node* head;
11
12 public:
13     DoublyLinkedList() : head(nullptr) {}
14
15     void insertEnd(int val) {
16         Node* newNode = new Node(val);
17         if (!head) {
18             head = newNode;
19             return;
20         }
21         Node* temp = head;
22         while (temp->next) temp = temp->next;
23         temp->next = newNode;
24         newNode->prev = temp;
25     }
26
27     // Standard approach: reverse second half, then merge alternately
28     void ReorderLinkedList() {
29         if (!head || !head->next) return;
30
31         // Step 1: Find middle with slow/fast pointers
32         Node* slow = head;
```

```
33     Node* fast = head;
34     while (fast && fast->next) {
35         slow = slow->next;
36         fast = fast->next->next;
37     }
38
39     // Step 2: Split list into two halves
40     Node* second = slow->next;
41     slow->next = nullptr;
42     if (second) second->prev = nullptr;
43
44     // Step 3: Reverse the second half
45     Node* prev = nullptr;
46     Node* curr = second;
47     while (curr) {
48         Node* nextNode = curr->next;
49         curr->next = prev;
50         curr->prev = nextNode;
51         prev = curr;
52         curr = nextNode;
53     }
54     second = prev; // new head of reversed half
55
56     // Step 4: Merge alternately
57     Node* first = head;
58     while (first && second) {
59         Node* temp1 = first->next;
60         Node* temp2 = second->next;
61
62         first->next = second;
63         second->prev = first;
64
65         if (temp1) {
66             second->next = temp1;
67             temp1->prev = second;
68         }
69
70         first = temp1;
71         second = temp2;
72     }
73 }
74
75 void printList() {
76     Node* temp = head;
77     while (temp) {
78         cout << temp->data << " ";
79         temp = temp->next;
80     }
81     cout << endl;
82 }
```


National University of Computer and Emerging Sciences
Lahore Campus

83 };